

Reflection

Building the Smart Office Document Assistant helped me understand the difference between having a working automation and having a system that people can actually use. In Part 1, n8n already handled the main business process: receiving documents, extracting information with AI, saving records to Google Sheets, storing files in Google Drive, and sending email notifications. What was missing was a simple application layer that allowed a user to interact with the automation without opening n8n.

The web application added that missing layer. A user can now upload a PDF, DOCX, or TXT file, see the processing result, search and filter processed documents, open a document, and complete a human review. The application also makes failures easier to understand. For example, unsupported files, large files, an unavailable n8n workflow, or a wrong API key produce clear messages instead of leaving the user with a technical error or an endless loading screen. Another important improvement was security. The browser does not receive the n8n shared secret because the application uses an Express server as a proxy.

Even with automation and AI, some decisions still require a person. The AI can extract fields such as customer, part number, urgency, quantity, and requested action, but it cannot always know whether the extracted information is correct or whether the business context is complete. This is why the review function is important. A user can mark a document as reviewed, add a name and note, and the decision is written back to Google Sheets. In a real sales environment, I would still want a human to review important or strategic requests before acting on them.

If the system had to process 1,000 documents per day, I think the first problems would be scale and reliability. Google Sheets is convenient for a small project, but it is not the ideal database for a high-volume production system. The n8n workflows could also become difficult to monitor if many executions were running at the same time. Large files, AI processing time, API limits, Google Drive operations, and email sending could all become bottlenecks. At that scale I would move the document records to a proper database, use queues for processing, add retry logic, monitoring, and better logging.

One thing I would build differently next time is the workflow architecture. For this project I created a separate webhook workflow based on the original Part 1 process. This worked, but it means that some logic exists in more than one workflow. If a field, AI extraction rule, or business rule changes, both workflows may need to be updated. A better design would move the shared document-processing logic into one reusable sub-workflow and let both the Google Drive trigger and the web application call the same process.

Overall, the most useful lesson from this project was that automation is not only about making tasks run automatically. A good solution also needs a clear interface, secure integration, understandable error handling, and a human decision point when judgment is still required.